

OpenAI 기반 바이브 코딩 실습 최소 가이드

송경모

2026-04-11

OpenAI 기반 바이브 코딩(Vibe Coding) 실습 가이드

대학교 교양 과목용 단계별 안내

0. 개요

이 문서는 AI 비전공자를 대상으로, OpenAI 플랫폼을 활용한 바이브 코딩(Vibe Coding)을 가장 쉬운 수준부터 고급 수준(Agentic AI 및 Harness Engineering)까지 단계적으로 안내한다.

1. 접근 절차 (Access & Setup)

1.1 최소 진입 (Level 0: No-code)

- 인터페이스: ChatGPT 웹
- 필요 SW: 없음
- 계정: OpenAI 계정
- 과금:
 - Free: 제한적 사용
 - Plus: 실습 가능 수준

활용

- 코드 생성
 - 간단한 수정
 - 개념 학습
-

1.2 준개발 환경 (Level 1: Light Coding)

- 인터페이스:
 - ChatGPT + 로컬 실행
- 필요 SW:
 - Python (Windows 설치)
 - VS Code (권장)

- 플러그인:
 - Python extension

과금 구조

1. Plus Plan에서도 Codex 사용 가능

→ 다만, 사용량 제한(quota 있음)이 있다. 대략 “주간 몇 번의 coding 세션 수준”, 초과 시 추가 크레딧 구매가 필요

- 가능
 - 코드 생성
 - Python 스크립트 작성
 - 간단한 agent 설계
 - 디버깅
- 제한(이런 제한을 풀려면 Pro요금제 상향)
 - 대형 프로젝트 (multi-file refactor)
 - 장시간 agent 실행
 - 반복적 자동화 실험

Codex는 별도 상품이 아니라 ChatGPT 유료 플랜(Plus, Pro, Business / Enterprise 등)에 포함된 기능이다

1.3 개발 환경 (Level 2: API 기반)

- 인터페이스:
 - Python + OpenAI API

이미 API key를 가지고 있다면 다음과 같은 식으로 모델을 “gpt-5.3-codex”로 설정하고 작업한다

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5.3-codex",
    input=" PDF 파일을 읽고 요약해주는 python 코드를 작성해줘."
    print(response.output[0].content[0].text)
```

- 필요:
 - API Key 를 받아놓고 .env 파일(프로젝트 폴더 안에 위치)에 별도로 기록해놓아야 한다.
 - 프로젝트 폴더 안에 python 가상환경 .venv 폴더가 생성되어 있어야 한다.
- 과금:
 - 이 방식은 토큰 사용량에 따라 별도 과금된다.

1.4 고급 환경 (Level 3: Agent / Harness)

- 인터페이스:
 - Python + Multi-agent 구조
- 필요:
 - 프레임워크 (LangChain 등 선택적)
 - 테스트 harness 구성

LangChain, LangGraph란 여러 회사의 모델에 통용될 수 있는, LLM을 보다 편리하게 다룰 수 있는 다양한 기능의 함수 라이브러리다.

2. 작업 시작 전 필수 정의 사항

2.1 기술 스택 확정

- 언어: Python (권장)
 - 실행 환경: Windows + Terminal
 - API 사용 여부 결정
-

2.2 과업 정의 프레임워크

다음 4가지를 반드시 명확히 한다:

(1) 목적 (Goal)

- 무엇을 만들 것인가
- 예: “PDF 요약 자동화 도구”

(2) 입력 (Input)

- 텍스트 / 파일 / API 등

(3) 출력 (Output)

- 텍스트 / JSON / 파일

(4) 처리 구조 (Process)

- 단계별 처리 흐름
-

2.3 Persona 및 역할 정의

예:

- “너는 Python 데이터 엔지니어다”
 - “너는 코드 리뷰 전문가다”
-

2.4 제약 조건 정의

- 실행 시간
 - 비용 제한
 - 정확도 요구 수준
-

3. 바이트 코딩의 범위 / 수준 / 한계

3.1 가능한 작업

기본

- 코드 생성
- 오류 수정
- 문서화

중급

- 파일 처리 자동화
- API 호출 시스템

고급

- Agent 구성
 - 멀티 스텝 reasoning
-

3.2 한계

- 완전 자동 시스템 설계는 불안정
 - 복잡한 상태 관리 어려움
 - 장기 실행 작업에서 오류 발생 가능
-

3.3 위험 요소

- hallucination (잘못된 코드)
 - dependency 충돌
 - 보안 문제 (API key 노출)
-

4. 작업 절차 (Workflow)

4.1 기본 구조

1. 문제 정의
 2. 프롬프트 설계
 3. 코드 생성
 4. 실행
 5. 디버깅
 6. 개선 반복
-

4.2 프롬프트 예시

You are a Python developer.

Task:

- Read a PDF file
- Extract text
- Summarize it

Requirements:

- Use PyMuPDF
- Output in bullet points
- Handle errors

Return:

- Complete Python script
-

4.3 프로젝트 폴더 구조

Level 1 에서는 다음과 같은 폴더 구조로 충분하다.

```
project/
├── main.py
├── utils/
├── data/
├── outputs/
├── prompts/
└── README.md
```

만약 Level 2 이상 API key를 받아서 하는 작업일 경우 폴더 구조는 다음과 같이 .env (API Key 기록 파일)과 .venv 폴더 (Python 실행 가상환경폴더)가 프로젝트 폴더 안에 포함되어 있어야 한다.

```
project/
├── .venv/           ← 필수 (실행 환경)
├── .env             ← 필수 (비밀키)
├── main.py
├── utils/
├── data/
├── outputs/
├── prompts/
├── requirements.txt
└── README.md
```

4.4 MD 파일 작성 원칙

필수 포함 항목

- 목적
- 입력/출력 정의
- 실행 방법
- 예시

작성 규칙

- 명확성 (ambiguous 금지)
- 재현 가능성
- 최소 단위 설명

4.5 디버깅 절차

1. 오류 메시지 확인
 2. AI에게 오류 전달
 3. 수정 요청
 4. 부분 테스트
 5. 전체 실행
-

4.6 검증 방법

- 입력-출력 테스트
 - edge case 테스트
 - 반복 실행 안정성 확인
-

4.7 배포 시 유의 사항

- API key 환경변수 처리
 - 로그 제거
 - 비용 관리
 - 라이선스 확인
-

5. Harness Engineering (최고 단계)

5.1 정의

Harness Engineering = AI 시스템을 안정적으로 실행하기 위한 실험/검증/통제 구조 설계

5.2 기본 구조

User Input
↓
Task Decomposition
↓
Agent Execution
↓
Validation
↓
Retry / Correction

5.3 핵심 구성 요소

(1) Task Decomposer

- 문제를 작은 단위로 분해

(2) Executor

- 각 단계 실행

(3) Validator

- 결과 검증

(4) Controller

- 흐름 관리
-

5.4 예시 구조

```
def run_pipeline(task):
    steps = decompose(task)

    results = []
    for step in steps:
        result = execute(step)

        if not validate(result):
            result = retry(step)

        results.append(result)

    return results
```

5.5 핵심 원칙

- deterministic control 확보
 - 실패 대비 구조 설계
 - 반복 실행 가능성
 - 비용 관리
-

5.6 교육용 최소 구현

학생 수준에서는 다음까지만 요구:

- 단일 agent
 - 단계 분리
 - 간단한 validation
-

6. 단계별 학습 로드맵

Level 0

- ChatGPT로 코드 생성

Level 1

- Python 실행

Level 2

- API 호출

Level 3

- 자동화 스크립트

Level 4

- Agent 구성

Level 5

- Harness Engineering
-

7. 결론

바이트 코딩은 “코드를 직접 작성하는 행위”가 아니라 “AI와 협력하여 시스템을 설계하는 행위”이다.

초보자는: → ChatGPT 기반 실습부터 시작

중급자는: → API + Python

고급자는: → Agent + Harness 구조

이 순서로 발전하는 것이 가장 효율적이다.